Machine Learning-Based Classification of LiDAR Point Clouds Using Gaussian Naïve Bayes, Support Vector Machine, Random Forest, and XGBoost

Course: Geospatial Machine Learning

Student: Tran Tran

Instructor: Dr. Heidar Rastiveis

University: Oregon State University

Date: March 5, 2026

Shared Results Folder:
https://oregonstate.box.com/s/vfa5w9dx828pz7yfiqrij765b0t32hhm

The shared folder contains the following materials:

- **Classified Original_Point_Cloud.las**
- **Classified testing point clouds**
- **Screenshots of classification results from CloudCompare**
- **All Python scripts used for feature extraction, model training, and evaluation**
- **Final report PDF**

Summary

This project investigates the use of machine learning techniques for classifying objects in LiDAR point cloud data. The goal of the study is to automatically classify points into four classes: ground, trees, cars, and buildings using geometric and radiometric features extracted from the point cloud.

First, a set of geometric features was extracted from the point cloud using local neighborhood analysis. These features describe the spatial structure of the surrounding points and include properties such as planarity, linearity, surface variation, and verticality. Radiometric features such as intensity and return number were also included in the feature set.

Two feature processing strategies were evaluated. The first approach used Sequential Forward Feature Selection (SFS) with Random Forest as base estimator to identify the most informative subset of features for classification. The second approach applied Principal Component Analysis (PCA) to reduce the dimensionality of the feature space while preserving 95% of the variance.

Four machine learning classifiers were tested and compared: Gaussian Naïve Bayes, Support Vector Machine, Random Forest, and Extreme Gradient Boosting (XGBoost). Each classifier was trained using labeled training data and evaluated on an independent test dataset. The models were evaluated using several quantitative metrics including confusion matrix, overall accuracy, Cohen's Kappa coefficient, and F1 score.

The results show that ensemble-based classifiers such as Random Forest and XGBoost, SVM generally achieve higher classification accuracy compared to simpler models such as Naïve Bayes. These models are better able to capture complex nonlinear relationships between point cloud features.

In addition to quantitative evaluation, qualitative analysis was performed by visualizing the classified point clouds in CloudCompare. The visual results demonstrate that machine learning methods can effectively classify large point cloud datasets and significantly reduce the need for manual labeling.

Overall, this study demonstrates that machine learning techniques provide an efficient and scalable approach for large-scale LiDAR point cloud classification.

Chapter 1: Methodology

1.1 Overview of the Methodology

This study investigates different techniques for point cloud classification. The goal is to distinguish between four object classes: ground, trees, cars, and buildings using geometric and radiometric information extracted from the point cloud.

The methodology consists of several stages. First, the raw LiDAR point clouds are processed to extract relevant features that describe the local geometric structure around each point. These features include geometric properties derived from the spatial distribution of neighboring points as well as radiometric attributes provided by the LiDAR sensor.

Once the features are extracted, two feature processing strategies are applied. The first strategy uses Sequential Forward Feature Selection (SFS) to identify the most informative subset of features for classification. The second strategy applies Principal Component Analysis (PCA) to reduce the dimensionality of the feature space while preserving 95% the variance in the data.

For each of the feature processing strategy, several machine learning classifiers are trained and evaluated. These classifiers include a Bayesian classifier, Support Vector Machine (SVM), Random Forest, and XGBoost. Each classifier is trained using the training dataset and evaluated using an independent test dataset.

Finally, the classification results are evaluated both quantitatively and qualitatively. Quantitative evaluation uses metrics such as confusion matrix, overall accuracy, Cohen's Kappa coefficient, and F1-score. Qualitative evaluation is performed by visualizing the classified test point clouds in CloudCompare. After that, we select the best model and perform classification on the original point cloud data of Corvallis

---

1.2 Classification Methods.

Bayesian Classifier

The Bayesian classifier or Gaussian Naïve Bayes is a probabilistic model based on Bayes' theorem. It assumes that features are conditionally independent given the class label and that the feature distributions follow a Gaussian distribution.

It is fast and efficient, making it suitable for large datasets. It also performs well when the independence assumption approximately holds. However, the assumption that all features are

independent may not always be valid in point cloud data, which can reduce classification performance.

Support Vector Machine (SVM)

Support Vector Machine is a model aiming to find an optimal hyperplane that separates data points belonging to different classes in the feature space.

One of the main strengths of SVM is its ability to handle high-dimensional data with different kernel tricks to map the input features into a higher-dimensional space. This enables SVM to handle non-linear boundaries

However, SVM can be computationally expensive when applied to large datasets and requires careful tuning of hyperparameters such as the kernel type and regularization parameter.

Random Forest

Random Forest is an ensemble learning method that combines multiple decision trees to improve classification performance. Each tree is trained on a random subset of the data and features, and the final prediction is determined by majority voting among all trees.

The main advantages of Random Forest include its robustness to noise, ability to handle high-dimensional data, and reduced risk of overfitting compared to a single decision tree. Additionally, Random Forest provides estimates of feature importance.

However, Random Forest models can become computationally expensive when a large number of trees are used, and the resulting models may be less interpretable.

XGBoost

XGBoost (Extreme Gradient Boosting) is a highly efficient implementation of gradient boosted decision trees. Unlike Random Forest, which builds trees independently, XGBoost builds trees sequentially, where each new tree attempts to correct the errors of previous trees.

The advantages of XGBoost are its ability to reduce overfitting and improve the model generalization with learning rate, regularization, handle missing data during training, and handle complex nonlinear relationships with tree structure.

Despite its advantages, XGBoost is computationally intensive, sensitive to noise and outliers, and can overfit on small datasets or with too many trees.

1.3 Feature Extraction in Point Clouds

Geometric Features

Geometric features describe the local structure of the point cloud around each point. These features are typically computed using the covariance matrix of neighboring points within a specified search radius. Eigenvalue decomposition of the covariance matrix provides several geometric descriptors such as

- **Linearity** – indicates how well points align along a line.
- **Planarity** – measures how well points lie on a plane.
- **Sphericity** – describes how evenly points are distributed in all directions.
- **Surface Variation** – represents the roughness of the surface.
- **Anisotropy** – indicates directional differences in point distribution.
- **Verticality** – measures how aligned a surface is with the vertical direction.

Additionally, the eigenvectors of the covariance matrix can be used to estimate surface normals (nx, ny, nz) that provides the orientation of surfaces.

Radiometric Features

Radiometric features originate from the LiDAR sensor capturing properties of the returned laser signal, such as:

- **Intensity** – the strength of the returned laser signal.
- **Return Number** – the order of the return in a multi-return pulse.

These features are particularly useful for distinguishing between objects with similar geometry but different surface materials.

1.4 Feature Selection

Because there are a lot of features from the data, feature selection aims to identify the most informative features for classification while removing redundant or irrelevant ones.

One common method is Sequential Forward Feature Selection. SFS is a greedy search algorithm that starts with an empty feature set and iteratively adds features that provide the greatest improvement in classification performance.

At each iteration, all remaining candidate features are evaluated, and the feature that results in the highest improvement in the evaluation metric is selected. This process continues until a predefined number of features is reached or no further improvement is observed.

1.5 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a dimensionality reduction technique that tries to maximize variance of the projected samples and removes the correlation between features.

It works by computing the eigenvectors and eigenvalues of the covariance matrix of the feature data. The eigenvectors define the directions of maximum variance in the data, while the corresponding eigenvalues indicate the amount of variance explained by each component.

The principal components are ordered based on the amount of variance they explain. By selecting only the first few components that capture most of the variance, the dimensionality of the dataset can be significantly reduced.

Chapter 2: Tests and Results

2.1 Dataset

The dataset consists of LiDAR point clouds stored in LAS format. There are two sets of data: training and testing. Both training and testing data contains four las files representing four object classes: ground, trees, cars, and buildings

Size:

Training	Training	Testing
Tree	40642	31598
Ground	81746	111130
Cars	1928	780
Buildings	24521	33023

Original Point Cloud size: 18854300

2.2 Parameters

Feature Extraction Parameters

Geometric features are computed using a **local neighborhood search** around each point. A spherical search radius (1 m) is used to identify neighboring points that contribute to the feature calculation.

Geometric Features:

- Eigenvalue Sum
- Omnivariance
- Eigenentropy
- Anisotropy

- Planarity
- Linearity
- PCA1
- PCA2
- Surface Variation
- Sphericity
- Verticality
- nx, ny, nz (surface normals)
- Number of neighbors

Radiometric Features

- Intensity
- Return number
- Scan angle rank

Feature Processing Parameters

Sequential Forward Selection (SFS)

Sequential Forward Selection was applied to identify the most informative subset of features. Parameters:

- Evaluation Metric: Classification Accuracy
- Cross Validation: 5 fold
- Number of selected features: 6
- Estimator: Random Forest

The best subset of features selected are: ['PCA1', 'sphericity', 'verticality', 'nx', 'nz', 'intensity]

Principal Component Analysis (PCA)

Principal Component Analysis was applied to reduce the dimensionality of the feature space with variance retained of 95%. With 20 features from radiometric features and geometric features, PCA is able to reduce to 10 components

Classifier Parameters

- **Bayesian Classifier** (var_smoothing=1.3)
- **Support Vector Machine (**RBF=1, C=1, Gamma = scale)
- **Random Forest:** (number of trees = 300, max_depth = None)
- **Xgboost:** n_estimators=500, max_depth=6, learning_rate = 0.01, subsample =0.9, colsample_by_tree = 0.9, objective=softmax, num_class =4, tree_method = hist, eval_metric = mlogloss

2.3 Quantitative Evaluation

Forward Selection – Bayesian:

True \ Predicted	**Ground**	**Trees**	**Cars**	**Buildings**
Ground	92403	10029	0	8698
Trees	19542	9984	0	2072
Cars	538	209	0	33
Buildings	23939	4190	0	4894

Forward Selection – Random Forest

True \ Predicted	**Ground**	**Trees**	**Cars**	**Buildings**
Ground	92526	11803	15	6786
Trees	14411	14274	5	2908
Cars	410	281	0	89
Buildings	18829	6710	8	7476

Forward XGBoost

True \ Predicted	**Ground**	**Trees**	**Cars**	**Buildings**
Ground	95484	10281	11	5354
Trees	15415	13853	0	2330
Cars	448	265	0	67
Buildings	20207	6115	0	6701

SVM

True \ Predicted	Ground	Trees	Cars	Buildings
Ground	94418	14691	0	2021
Trees	16561	13794	0	1243

Forward XGBoost

True \ Predicted	Ground	Trees	Cars	Buildings
Cars	399	366	0	15
Buildings	22779	6311	0	3933

PCA

GaussianNB	**Ground**	**Trees**	**Cars**	**Buildings**
Ground	83154	10075	0	17901
Trees	18172	10524	0	2902
Cars	477	233	0	70
Buildings	20470	4154	0	8399
xgboost	**Ground**	**Trees**	**Cars**	**Buildings**
Ground	100689	8221	1	2219
Trees	19819	10273	12	1494
Cars	419	305	2	54
Buildings	24220	4352	6	4445
xgboost	**Ground**	**Trees**	**Cars**	**Buildings**
Ground	97706	9385	307	3732
Trees	18171	10101	605	2721
Cars	391	309	4	76
Buildings	21678	5220	200	5925
SVM	**Ground**	**Trees**	**Cars**	**Buildings**

GaussianNB	Ground	Trees	Cars	Buildings
True \ Predicted				
Ground	96080	13115	0	1935
Trees	17516	12765	0	1317
Cars	411	344	0	25
Buildings	22073	6452	0	4498

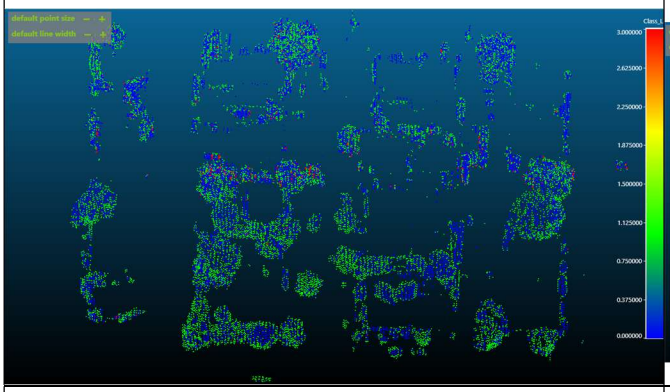
Selected features (SFS):

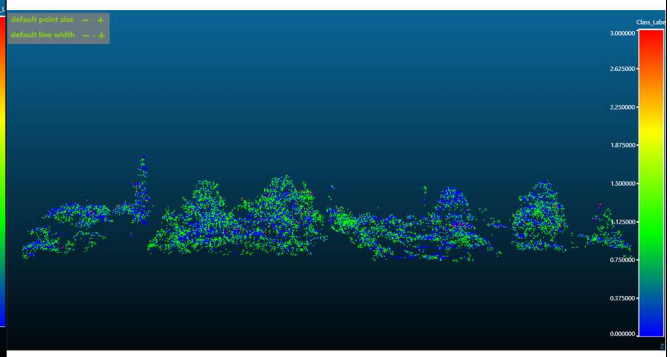
SVM_rbf => ['PCA1', 'sphericity', verticality', 'nx', 'nz', 'intensity']

Feature Strategy	Method	Accuracy	Kappa	F1 Score	Training Time (s)	Testing Time (s)
Forward Selection	Bayesian	0.61	0.17	0.32	0.44	0.14
Forward Selection	SVM	0.64	024	0.35	703.s	594.5s
Forward Selection	Random Forest	0.65	0.29	0.38	41.9	22.9
Forward Selection	XGBoost	0.66	0.3	0.38	61.4	3.3
PCA (95%)	Bayesian	0.58	0.17	0.34	1.16s	0.38s
PCA (95%)	SVM	0.65	0.24	0.34	6978.88	1447s
PCA (95%)	Random Forest	0.64	0.24	0.35	87.6	7.73
PCA (95%)	XGBoost	0.65	0.23	0.345	39.67s	2.51s

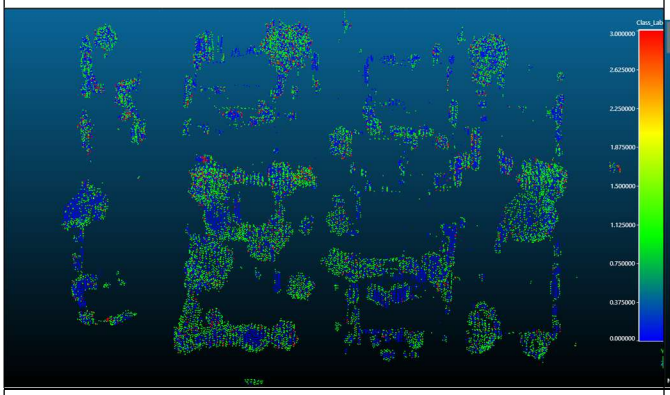
- **2-4: Qualitative Results**

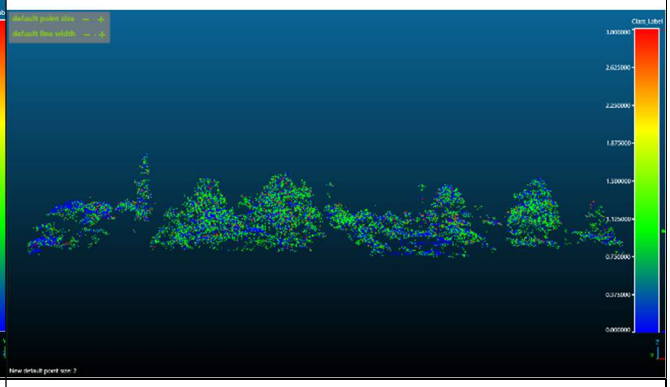
SVM



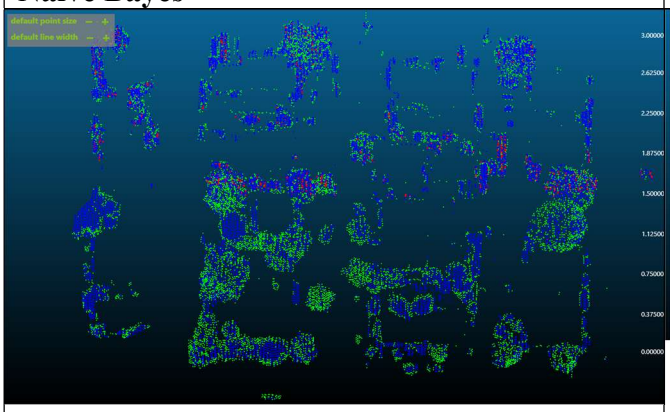


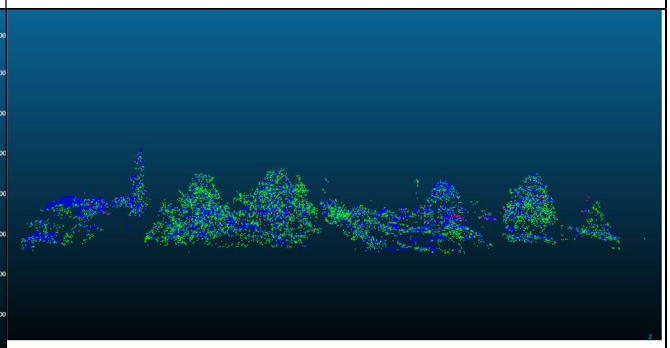
Random Forest



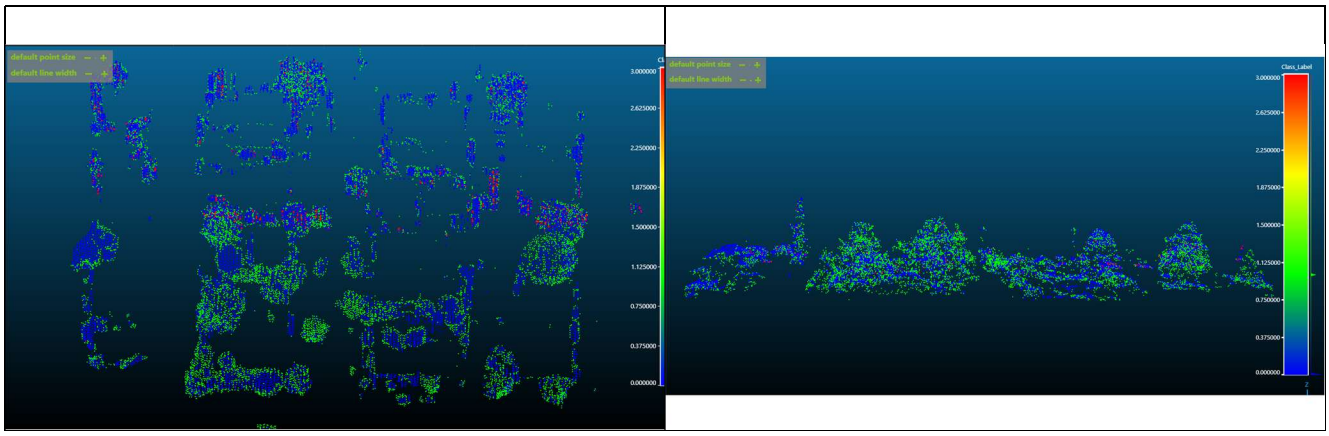


Naïve Bayes



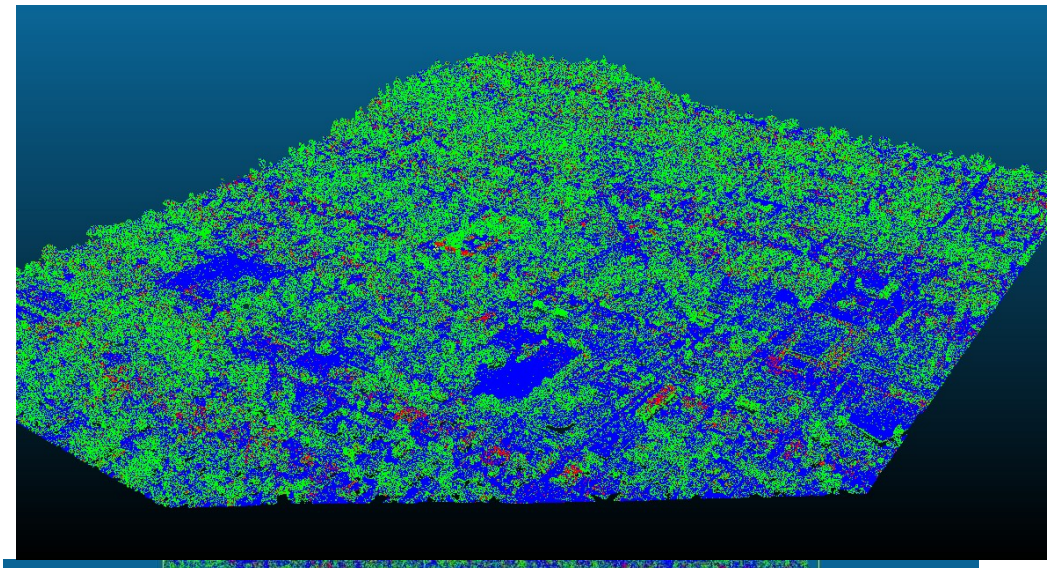


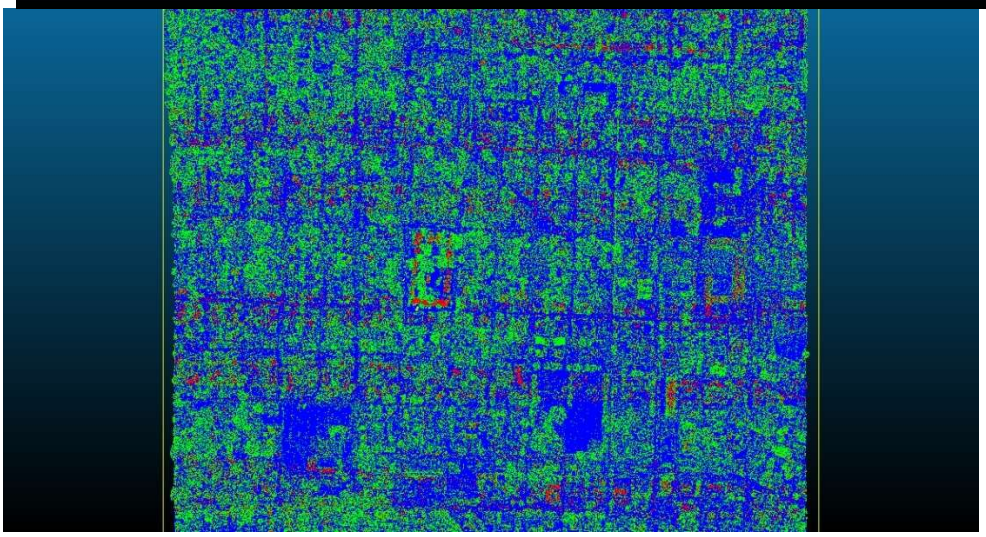
xgboost



- .2-5- Results of the classification on the Original_Point_Cloud.las.

Blue for Ground, Green for Tree, Yellow for Car, and Red for Buildings





Chapter 3: Discussion:

3.1 Processing time: With large point cloud data, it would be very hard and slow to extract points for each class, especially when objects overlap. It may take several hours for medium size dataset. Applying machine learning would reduce the computation time to the order of minutes once the model is trained. Furthermore, the time of training doesn't require full labor time and attention from human. It also improves the ability for the classification and any detection to be automated.

3.2 Best Classifier Performance: XGBoost. This classifier achieved the highest values for overall accuracy, Cohen's Kappa coefficient, and F1 score. The strong performance of this classifier can be attributed to its ability to capture complex nonlinear relationships between features. Point cloud data contains irregular structures and noisy measurements.

Important hyperparameters are number of trees controlling the number of decision trees, maximum tree depth, and learning rate. Maximum tree depth limits the depth of the tree to avoid overfitting

Classifier Strengths and Weaknesses

Bayesian Classifier: The Bayesian classifier is computationally efficient and easy to implement. It is useful to combine with Sequential Feature Selector to choose an optimal subset of features. However, its main limitation is the assumption that features are independent which doesn't hold trues in point cloud geometric feature

Support Vector Machine: Support Vector Machines are effective in high-dimensional feature spaces and can model nonlinear decision boundaries using kernel functions. However, SVM can be computationally expensive.

Random Forest: Random Forest classifiers are robust and handle noisy data well. They can model complex relationships between features and provide reliable classification results without extensive hyperparameter tuning. However, decisions are trained independently instead of sequentially like XGBoost so it doesn't iteratively fixing prediction errors.

XGBoost: XGBoost can also capture complex nonlinear patterns in the data. It also includes built-in regularization mechanisms that help prevent overfitting. However, XGBoost hyperparameters are more complicated than Random Forst

---

3.3 Challenges Encountered

Data Issues: LiDAR point clouds may contain noise, missing points, and outliers that can affect feature extraction and classification performance. In some cases, geometric features produced undefined values (NaNs) due to insufficient neighboring points within the search radius. These values were handled by replacing NaNs with zeros.

Classifier Limitations: SVM takes very long to compute compared to other datasets. Compared to xgboost, it's interesting that Random Forest takes fast training but longer to test

Certain classifiers showed limitations when applied to this dataset. For example, the Bayesian classifier sometimes misclassified objects with similar geometric structures, such as trees and buildings, because the independence assumption between features does not hold in practice.

SVM models were computationally slower compared to other classifiers.

> **Some classifiers also struggled to distinguish between classes with similar geometric characteristics, particularly between vertical structures such as trees and buildings.**

Hyperparameter Tuning:

Hyperparameter tuning yielded better results. For example, I tried a search radius to be 0.5 and the results are worse than search radius =1. Sometimes, I adjusted the hyperparameter to reduce computational complexity, for example, setting random forest to 6 instead of None, or choosing 6 features instead of 4 and 5 because they produce a better result.

Scalability Issues

The most computationally expensive stage is feature extraction, as computing geometric features requires searching for neighboring points for each point in the dataset. This process can be improved by using spatial index to search for point more effectively, or using parallel processing.

3.4 Additional Observations

Training and Testing Data Separation: The dataset was divided into training and testing subsets to evaluate model performance objectively and check for the model's ability to generalize to different dataset. However, training dataset is smaller then testing set which should be the opposite. Training set should be more diverse and abundant.

Class Imbalance: There is a very heavy class imbalance between ground and tree vs car vs building. Ground and trees contain much more data than buildings which contain more data than cars. As a result, none of the classifiers were able to detect cars adequately.

Chapter 4: Conclusion:

4.1. Summary of Results: Overall, overall accuracy ranges from 60% and 66% and most of accuracy detected is for the ground, tree and building data meanwhile none of the classifiers are able to detect cars. Out of the four models, Gaussian NB is the least computationally expensive but it also gives the least accurate result which is not too poor compared to other classified. SVM is the most computationally expensive given that performance is not any higher than xgboost or random forest.

4-2: Future Work: Future work could include having more than one set of geometric features computed with different search radius to capture both detailed and general patterns. Other

advanced deep learning methods such as PointNet or transformer can be applied to improve point cloud data without the need to extract features.